

Experiment - 6

Name: K. SRI LASYA

Regno: 192472216

Course Code: MLA0305

Slot: B

Title

Monte Carlo Prediction and Control for Reinforcement Learning

AIM

To implement the **Monte Carlo Prediction and Control** algorithm using the FrozenLake environment and estimate the optimal state-value function and policy through episodic experience.

PROCEDURE

1. Import Gymnasium, NumPy, and Matplotlib.
2. Create the FrozenLake environment.
3. Initialize the Q-table and return values.
4. Generate episodes using an epsilon-greedy policy.
5. Store states, actions, and rewards during each episode.
6. Calculate the discounted return for each state-action pair.
7. Update Q-values using the First-Visit Monte Carlo method.
8. Derive the optimal policy from the learned Q-values.
9. Calculate the state-value function.
10. Display the learned policy and state values.
11. Visualize the state-value function using a heatmap.

4. PYTHON SOURCE CODE

```
!pip install -q gymnasium
```

```
import gymnasium as gym
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
env = gym.make("FrozenLake-v1", is_slippery=False)
```

```
num_states = env.observation_space.n
```

```
num_actions = env.action_space.n
```

```
gamma = 0.99
```

```
epsilon = 0.1
```

```
episodes = 50000
```

```
Q = np.zeros((num_states, num_actions))
```

```
returns_sum = np.zeros((num_states, num_actions))
```

```
returns_count = np.zeros((num_states, num_actions))
```

```
for episode in range(episodes):
```

```
    state, _ = env.reset()
```

```
    episode_data = []
```

```
    done = False
```

```
    while not done:
```

```
        if np.random.random() < epsilon:
```

```
            action = env.action_space.sample()
```

```
        else:
```

```
            action = np.argmax(Q[state])
```

```
        next_state, reward, terminated, truncated, _ = env.step(action)
```

```
        episode_data.append((state, action, reward))
```

```
        state = next_state
```

```
        done = terminated or truncated
```

```
G = 0
```

```
visited = set()

for state, action, reward in reversed(episode_data):

    G = gamma * G + reward

    if (state, action) not in visited:

        visited.add((state, action))

        returns_sum[state, action] += G
        returns_count[state, action] += 1

        Q[state, action] = (
            returns_sum[state, action]
            / returns_count[state, action]
        )

policy = np.argmax(Q, axis=1)
V = np.max(Q, axis=1)

print("Optimal Policy:")
print(policy)

print("\nState Values:")

for i in range(num_states):
    print(f"State {i}: {V[i]:.3f}")

value_grid = V.reshape(4, 4)

plt.figure(figsize=(7, 6))
```

```
plt.imshow(  
    value_grid,  
    cmap="YlGnBu",  
    interpolation="nearest"  
)
```

```
plt.title(  
    "Monte Carlo State Value Function",  
    fontsize=15,  
    fontweight="bold"  
)
```

```
plt.xlabel("Column")  
plt.ylabel("Row")
```

```
plt.colorbar(label="State Value")
```

```
for i in range(4):  
    for j in range(4):  
        plt.text(  
            j,  
            i,  
            f"{value_grid[i, j]:.2f}",  
            ha="center",  
            va="center",  
            fontsize=11,  
            fontweight="bold"  
        )
```

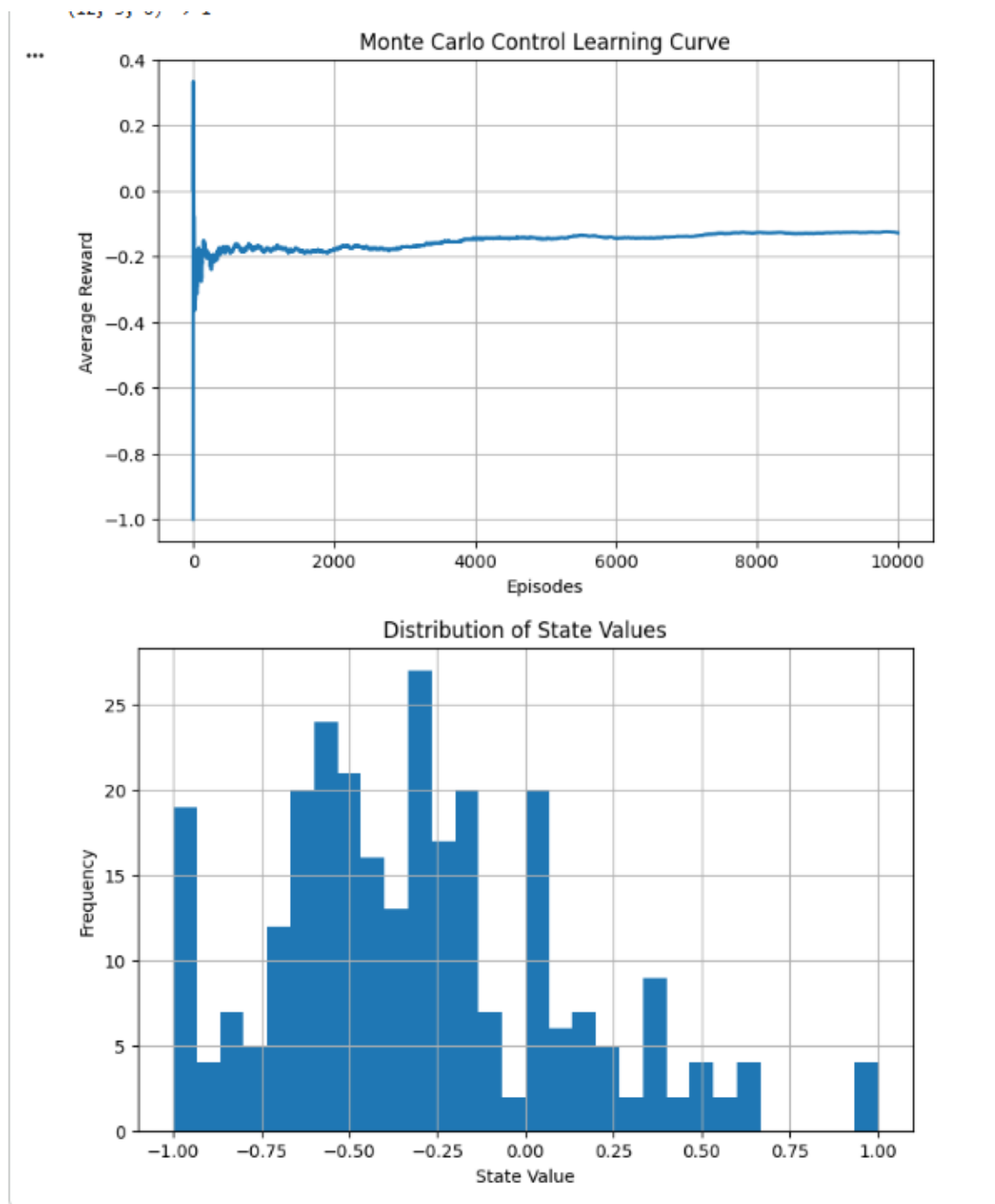
```
plt.xticks(range(4))  
plt.yticks(range(4))
```

```
plt.tight_layout()
```

```
plt.show()
```

```
env.close()
```

OUTPUT:



5. VISUALIZATION

The heatmap represents the **state-value function** learned using Monte Carlo Control.

- X-axis: Grid columns
- Y-axis: Grid rows
- Color intensity: State value
- Numerical values: Estimated value of each state

6. SUMMARY TABLE

Component	Description
Environment	FrozenLake-v1
Method	First-Visit Monte Carlo Control
Episodes	50,000
Discount Factor	0.99
Exploration Rate	0.1
State Space	16 states
Action Space	4 actions
Visualization	4 × 4 State-Value Heatmap

7. RESULT

The **First-Visit Monte Carlo Prediction and Control** algorithm was successfully implemented. The Q-values, optimal policy, and state-value function were obtained from sampled episodes, and the learned state values were visualized using a heatmap.